PROJETO INTEGRADOR II

AVALIAÇÃO 3 – ARQUITETURA DISTRIBUÍDA: BANCO DE DADOS, API REST, DOCUMENTAÇÃO E INTEGRAÇÃO DO APLICATIVO

1 INFORMAÇÕES DA EQUIPE

Nomes do Líder e dos Integrantes da Equipe:

1. Thalys Rocha Rodrigues (20242158060070) – líder
2. Luiz Ailton de Araujo Souza (20242158060041)
3. Paulo Cesar Lima da Silva (20242158060054)

2 IDENTIFICAÇÃO DO PROJETO

Nome da aplicação: Zé Resolve

Tipo de projeto: Aplicativo móvel de intermediação de serviços locais.

Público-alvo: Clientes que necessitam contratar serviços e prestadores autônomos de diferentes áreas.

Repositório (código-fonte): https://github.com/ThalysRD/Ze-Resolve

Documentação OpenAPI (arquivo):
https://github.com/ThalysRD/Ze-Resolve/blob/main/docs/openapi.json

Arquitetura e justificativas:
https://github.com/ThalysRD/Ze-Resolve/blob/main/docs/arquitetura.md

Protótipo no Figma: https://www.figma.com/design/OmP9I8Z8iEBPy6duwa1mZT/App-projeto-integrador-2?node-id=0-1&t=oDYtz27xaVNoQlmy-1

3 OBJETIVO DA AVALIAÇÃO 3

Na Avaliação 2, o Zé Resolve tinha as telas implementadas em Flutter, mas os dados e o login eram mantidos em memória pelo próprio aplicativo, sem servidor nem persistência. Nesta etapa, o sistema evoluiu para uma arquitetura distribuída: o aplicativo passou a se comunicar com um servidor próprio por uma API HTTP/REST, e esse servidor concentra as regras de negócio, a persistência em um banco de dados relacional e o controle de acesso.

O trabalho foi organizado em cinco frentes, que correspondem às seções deste documento: modelagem e persistência dos dados (seção 5), desenvolvimento da API REST (seção 6), documentação com OpenAPI (seção 7), integração do aplicativo ao servidor (seção 8) e identificação e controle de acesso (seção 9). Todo o código está no repositório indicado na seção 2.

4 ARQUITETURA PROPOSTA

A arquitetura segue o fluxo Aplicativo → API → Banco de Dados. O aplicativo Flutter contém apenas a interface e um módulo de comunicação; ele nunca acessa o banco. A API, escrita em Python com FastAPI, recebe as requisições, valida os dados, verifica a identidade e as permissões do usuário, aplica as regras de negócio e grava ou consulta o banco por meio do

ORM SQLAlchemy. O chat usa, além do REST, um canal WebSocket autenticado, que confirma cada mensagem somente depois de gravá-la no banco.

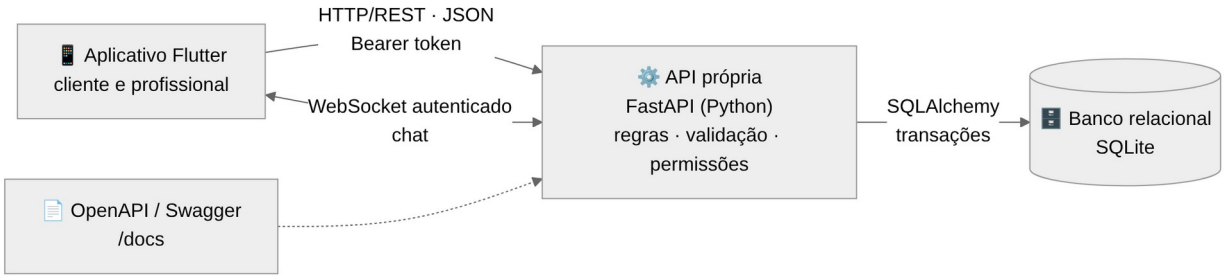


Figura 1 – Arquitetura do Zé Resolve: Aplicativo → API → Banco de Dados.

Tabela 1 – Componentes da arquitetura.

Camada	Tecnologia	Responsabilidade
Interface (cliente)	Flutter / Dart (Android)	Telas do cliente e do profissional; consome a API por HTTP/REST e WebSocket; armazena o token de sessão de forma segura.
Servidor (API)	Python, FastAPI, Uvicorn	Rotas REST, validação (Pydantic), regras de negócio, autenticação, permissões, CORS e documentação OpenAPI.
Persistência	SQLite + SQLAlchemy	Tabelas, relacionamentos e restrições de integridade; transações; dados mantidos em arquivo.
Integrações opcionais	Firebase Cloud Messaging, SMTP	Notificações push e envio de email; o sistema funciona sem elas.

5 MODELAGEM E PERSISTÊNCIA DE DADOS

5.1 Entidades

As entidades foram identificadas a partir dos casos de uso definidos nas avaliações anteriores: cadastro e login, catálogo de profissionais por categoria, agenda de horários, solicitação e acompanhamento do serviço, conversa entre as partes e avaliação ao final.

Tabela 2 – Entidades principais.

Entidade (tabela)	Descrição e campos principais
`users`	Usuário do sistema: nome, email (único), telefone, perfil (`cliente` ou `profissional`) e hash bcrypt da senha.
`auth_tokens`	Sessões de login: hash do token, usuário, emissão, expiração e revogação.
`categories`	Categorias de serviço (Eletricista, Encanador, Pintor etc.), com nome único.
`professionals`	Perfil profissional ligado a um usuário: categoria, descrição, cidade, valor fixo da visita (centavos) e localização aproximada.
`availability_slots`	Horários publicados pelo profissional: início e fim, ativo ou arquivado.
`service_requests`	Solicitação de serviço: cliente, profissional, horário reservado, título, descrição, endereço, valor da visita e estado.
`reviews`	Avaliação de um serviço concluído: nota de 1 a 5 e comentário.
`messages`	Mensagens trocadas entre cliente e profissional em uma solicitação.
`addresses`	Endereços salvos pelo usuário (dados privados cifrados).

Há ainda tabelas de apoio para controle de leitura de mensagens, avisos de mudança de etapa, dispositivos e fila de notificações push, limite de tentativas de login e auditoria de conta. O modelo completo, com tipos e índices, está em `docs/schema.sql` e em `docs/arquitetura.md`.

5.2 Relacionamentos

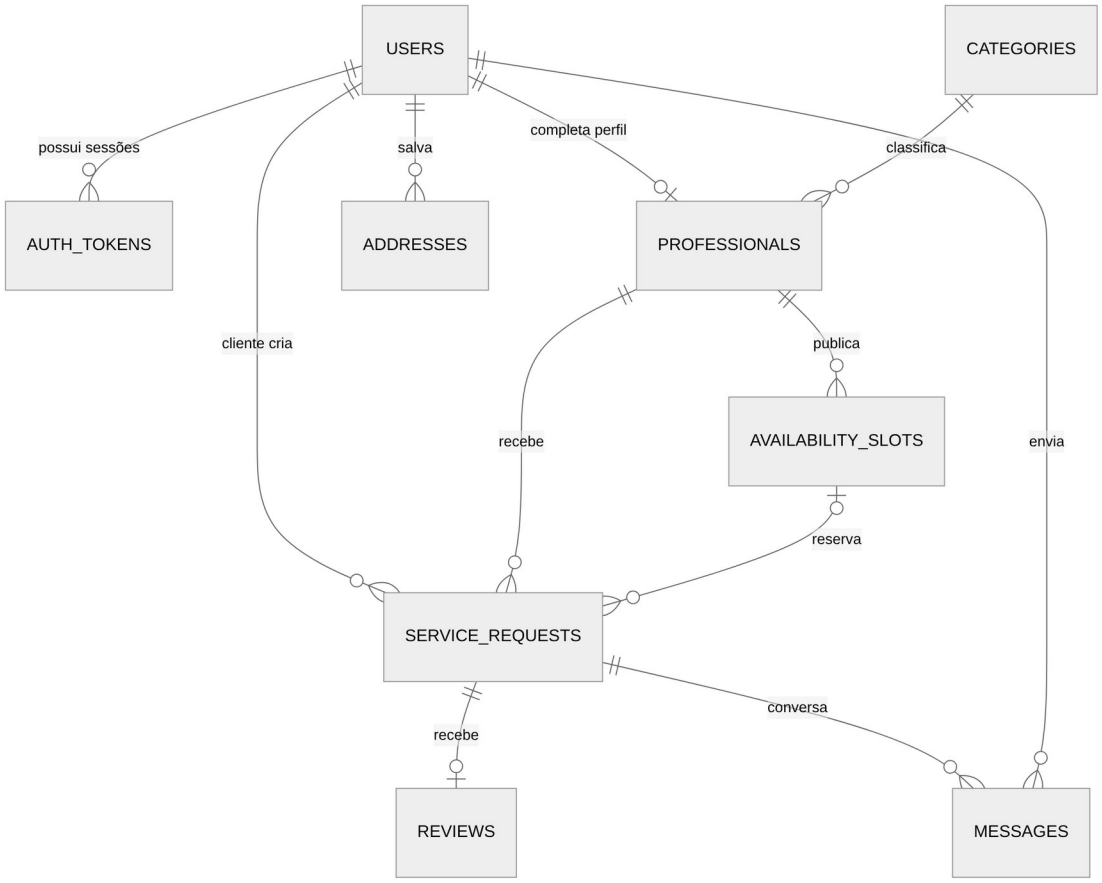


Figura 2 – Diagrama entidade-relacionamento das entidades principais.

5.3 Regras de integridade

As regras são aplicadas em duas camadas: no banco, por chaves estrangeiras e restrições, e na API, por validações e regras de negócio. No banco:

- **Chaves estrangeiras** ativadas em toda conexão (`PRAGMA foreign_keys=ON`), impedindo registros órfãos.
- **Unicidade:** email do usuário, nome da categoria, uma única avaliação por solicitação e um identificador por mensagem enviada, que impede duplicação em reenvios.
- **CHECK:** perfil restrito a `cliente` ou `profissional`; cliente e profissional diferentes na mesma solicitação; nota entre 1 e 5; fim do horário posterior ao início; duração entre 30 e 480 minutos; coordenadas dentro dos limites válidos.
- **Índice único parcial** que impede duas reservas ativas no mesmo horário, mesmo com pedidos simultâneos.
- **Sem exclusão física:** a API não possui rotas DELETE. Sessões são revogadas, horários e endereços são arquivados e pedidos mantêm o estado final, preservando o histórico.

5.4 Escolha do banco de dados

Foi escolhido o **SQLite**, um banco de dados relacional que oferece transações ACID, chaves estrangeiras, índices e restrições CHECK e UNIQUE, ou seja, todos os recursos necessários para as regras de integridade descritas acima. Ele grava os dados em um único arquivo e já acompanha o Python, então não exige instalar nem iniciar um serviço de banco separado. Isso simplifica a execução do projeto em qualquer computador, inclusive pelo professor, e atende ao volume de uma demonstração ou piloto.

O acesso é feito pelo ORM **SQLAlchemy**, que define as entidades como classes Python e centraliza as consultas na API. O modo WAL e as transações coordenam leituras e gravações concorrentes. A limitação conhecida é que o SQLite não é indicado para vários servidores escrevendo ao mesmo tempo; para uma operação pública em maior escala, um banco cliente-servidor como o PostgreSQL seria o próximo passo.

5.5 Dados iniciais e persistência

O comando de seed cria as categorias e dados fictícios para testes: profissionais com horários publicados, uma solicitação pendente e uma concluída com avaliação. Ele pode ser executado novamente sem duplicar dados. São criadas três contas de demonstração, todas com a senha `Demo12345!`:

Tabela 3 – Contas de demonstração criadas pelo seed.

Perfil	Email
Cliente	`cliente@example.com`
Profissional (Eletricista)	`profissional@example.com`
Profissional (Encanador)	`encanador@example.com`

O banco fica no arquivo `backend/data/apresentacao.db`, criado automaticamente na primeira execução. Como os dados ficam em arquivo, eles permanecem disponíveis após reiniciar o servidor. Isso foi verificado a partir de um clone limpo do repositório: após o seed e a reinicialização da API, o login com as contas de demonstração continuou funcionando.

6 API HTTP/REST

6.1 Recursos, caminhos e formatos

A API expõe 43 operações, organizadas por recurso. Todas recebem e retornam **JSON**, usam IDs inteiros, datas no formato ISO 8601 em UTC e valores monetários em centavos. Listas são paginadas com os parâmetros `limit` (1 a 100) e `offset`. A Tabela 4 resume os recursos; a lista completa de rotas está no Apêndice A.

Tabela 4 – Recursos da API e operações principais.

Recurso	Operações principais	Acesso
Autenticação	`POST /auth/register`, `POST /auth/login`, `POST /auth/logout`	Pública (logout exige token)
Usuário e endereços	`GET/PATCH /users/me`, `GET/POST /users/me/addresses`	Usuário autenticado (somente os próprios dados)
Categorias	`GET /categories`	Pública

Recurso	Operações principais	Acesso
Profissionais	`GET /professionals` (filtros: categoria, cidade, proximidade, disponibilidade), `GET /professionals/{id}`	Pública
Perfil e agenda do profissional	`GET/PUT /professionals/me`, `GET/POST /professionals/me/availability`	Somente profissional
Solicitações	`POST /requests`, `GET /requests`, `GET /requests/{id}`, `PATCH /requests/{id}/status`	Criação: cliente. Consulta e etapas: participantes
Avaliações	`POST /requests/{id}/review`, `GET /professionals/{id}/reviews`	Criação: cliente, após conclusão
Mensagens	`GET/POST /requests/{id}/messages` e WebSocket	Participantes da solicitação

Exemplo de criação de solicitação pelo cliente (o servidor obtém o cliente pelo token e deriva horário e duração do horário escolhido):

```
POST /requests
Authorization: Bearer TOKEN
Content-Type: application/json

{ "professional_id": 2, "slot_id": 5, "expected_visit_fee_cents": 8000,
  "title": "Instalação de tomadas",
  "description": "Preciso instalar três tomadas na sala.",
  "address": "Rua das Flores, 123 — Natal, RN" }

201 Created
{ "id": 3, "client_id": 1, "professional_id": 2, "slot_id": 5,
  "visit_fee_cents": 8000, "scheduled_at": "2026-10-28T17:00:00Z",
  "duration_minutes": 60, "status": "pendente", ... }
```

6.2 Estados do fluxo principal

O fluxo principal é a solicitação de serviço. Ela nasce **pendente**, ocupando o horário escolhido, e só avança pelas transições permitidas na Figura 3. Cada transição é validada no servidor de acordo com o perfil de quem a solicita e com o estado atual registrado no banco.

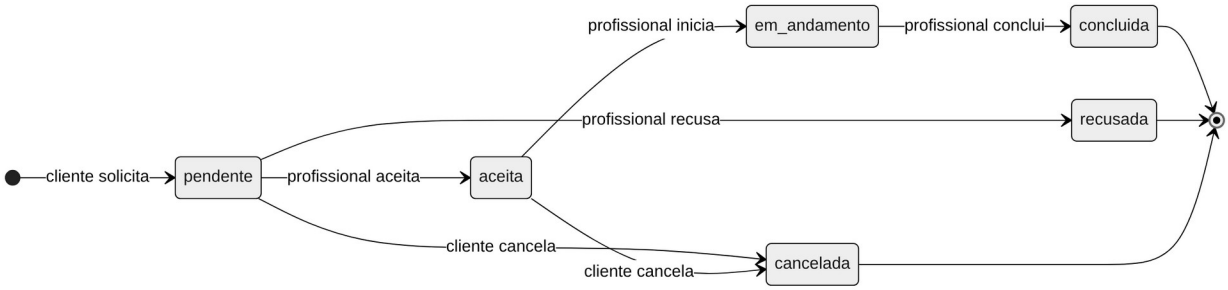


Figura 3 – Estados da solicitação de serviço e transições permitidas.

Tabela 5 – Transições permitidas.

Transição	Quem pode executar	Observação
pendente → aceita	Profissional atribuído	Confirma o atendimento no horário reservado.

Transição	Quem pode executar	Observação
pendente → recusada	Profissional atribuído	Libera o horário para outros clientes.
pendente/aceita → cancelada	Cliente que criou	Libera o horário.
aceita → em_andamento	Profissional atribuído	Início do atendimento.
em_andamento → concluida	Profissional atribuído	Habilita a avaliação pelo cliente.
concluida, recusada, cancelada	—	Estados finais; não permitem novas transições.

Uma transição fora dessa tabela retorna **409**; um perfil que não pode executar a etapa recebe **403**. Pedidos pendentes não respondidos expiram após o prazo configurado. A alteração compara o estado anterior gravado no banco, o que protege contra duas ações simultâneas sobre o mesmo pedido.

6.3 Validação e códigos HTTP

Os dados recebidos são validados no servidor por modelos Pydantic, que verificam tipo, tamanho, formato, intervalos e campos obrigatórios e rejeitam campos desconhecidos. Regras de negócio, como horário futuro e livre ou valor da visita igual ao exibido, também são verificadas. As respostas usam códigos HTTP coerentes. Os exemplos da Tabela 6 foram obtidos da API em execução:

Tabela 6 – Códigos HTTP utilizados, com respostas reais da API.

Código	Situação	Exemplo de resposta
200 / 201 / 204	Sucesso (consulta, criação, operação sem corpo)	`{"id": 3, "status": "pendente", ...}`
401	Sem token, token inválido/expirado ou login incorreto	`{"detail": "Autenticação necessária ou sessão expirada."}`
403	Perfil sem permissão (ex.: cliente aceitando o próprio pedido)	`{"detail": "Seu perfil não pode executar esta etapa."}`
404	Recurso inexistente	`{"detail": "Solicitação não encontrada."}`
409	Transição de estado inválida, duplicata ou conflito	`{"detail": "Transição de estado não permitida."}`
422	Dados inválidos (indica campo e motivo)	`{"detail": [{"loc": ["body","password"], "msg": "String should have at least 8 characters", "type": "string_too_short"}]}`
429	Excesso de tentativas de login/cadastro	Mensagem genérica e cabeçalho `Retry-After`

6.4 CORS

O CORS está configurado na API com o middleware do FastAPI. As origens permitidas vêm da variável `CORS_ORIGINS` (por padrão `http://localhost:8080` e `http://127.0.0.1:8080`, para um painel web local), com os métodos `GET`, `POST`, `PUT` e `PATCH` e os cabeçalhos `Authorization` e `Content-Type`. Em uma requisição de verificação (preflight), a API responde `access-control-allow-origin: http://localhost:8080`. O CORS é uma regra dos navegadores, então o aplicativo Android não depende dele; a configuração existe para que uma interface web possa consumir a mesma API, e outras origens podem ser liberadas pela variável de ambiente, sem alterar o código.

6.5 REST em relação ao SOAP

Tabela 7 – Comparação entre REST e SOAP para o Zé Resolve.

Aspecto	REST (adotado)	SOAP
Modelo	Recursos identificados por URLs e manipulados pelos métodos HTTP (GET, POST, PUT, PATCH).	Operações remotas encapsuladas em envelopes XML, normalmente via POST.
Formato	JSON, leve e convertido nativamente pelo Dart e pelo Python.	XML obrigatório, mais verboso e com processamento mais pesado no celular.
Contrato	OpenAPI, gerado automaticamente pelo FastAPI e testável no Swagger.	WSDL, mais rígido e voltado a ferramentas corporativas.
Erros	Códigos HTTP padronizados (401, 403, 404, 409, 422).	SOAP Fault dentro do envelope, geralmente com HTTP 500.
Segurança	HTTPS e token Bearer.	WS-Security, completo, porém complexo de configurar.
Adequação ao projeto	Simples de consumir pelo app e por um futuro painel web; cache e ferramentas HTTP comuns.	Indicado para integrações corporativas com transações e contratos formais.

O REST foi escolhido porque o Zé Resolve é um aplicativo móvel que faz operações simples sobre recursos bem definidos (profissionais, horários, solicitações, mensagens). O JSON é leve para redes móveis e é tratado nativamente pelo Flutter, e o contrato OpenAPI permite que outra interface, como um painel web, use a API de forma independente. Os recursos do SOAP, como WS-Security e contratos WSDL, acrescentariam complexidade sem benefício para esse cenário. Para a conversa em tempo real, onde o REST não é adequado, foi usado WebSocket.

7 DOCUMENTAÇÃO DA API COM OPENAPI

A documentação é gerada automaticamente pelo FastAPI a partir do próprio código e segue a especificação OpenAPI 3.1. Ela pode ser consultada de três formas:

- **Swagger UI interativo** em `http://127.0.0.1:8000/docs`, com a API em execução, onde é possível testar cada rota pelo botão **Try it out**;
- **Arquivo JSON** em `http://127.0.0.1:8000/openapi.json`;
- **Arquivo versionado** `docs/openapi.json` no repositório, disponível sem executar o projeto (link na seção 2).

O contrato documenta todas as 43 operações, cada uma com:

- resumo e descrição da finalidade, de quem pode chamá-la e das regras aplicadas;
- parâmetros de caminho e de consulta, com tipos e limites (por exemplo, `limit` entre 1 e 100);
- esquemas dos corpos de requisição e resposta, com o tipo de cada campo e as regras de validação (tamanho mínimo e máximo, formato de email, valores permitidos, campos obrigatórios);
- exemplos de requisição e de resposta de sucesso e de cada erro (400, 401, 403, 404, 409, 422, 429);

- mecanismo de autenticação: o esquema `HTTPBearer` está declarado, e as 30 rotas que exigem login aparecem com um cadeado. Para testá-las, faz-se login em `POST /auth/login` e cola-se o `access_token` no botão **Authorize**.

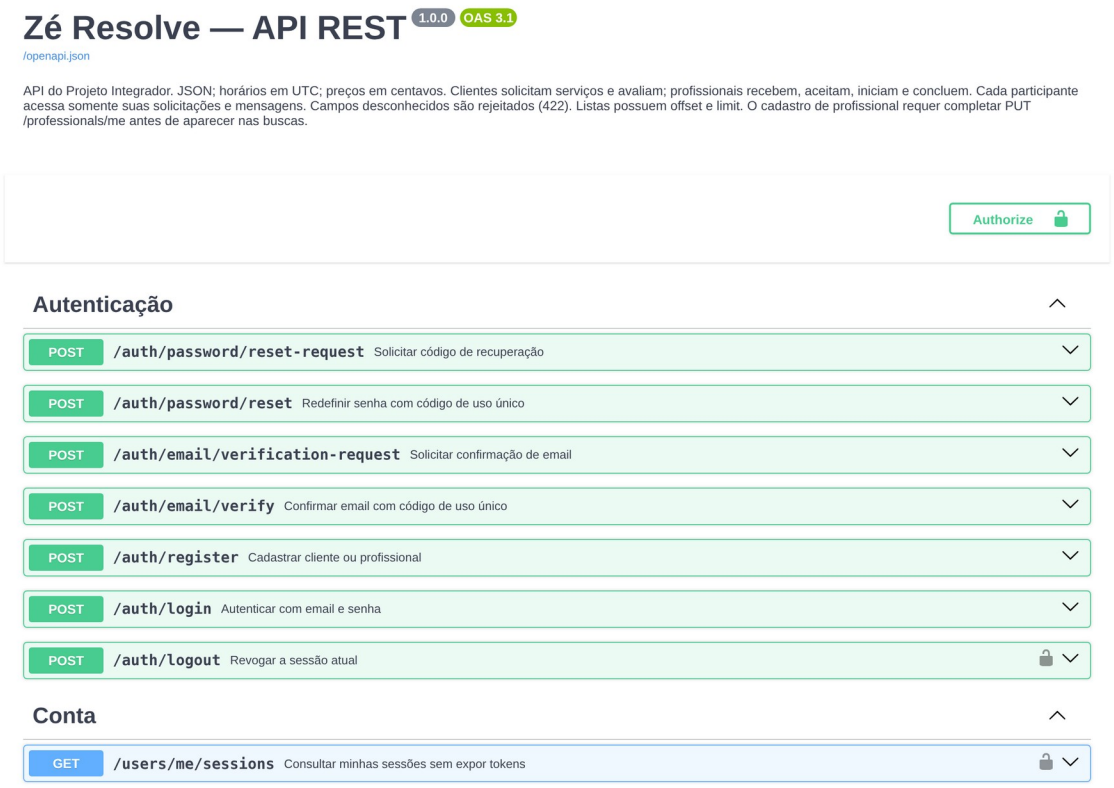


Figura 4 – Swagger UI da API do Zé Resolve, com o botão Authorize e as rotas agrupadas.

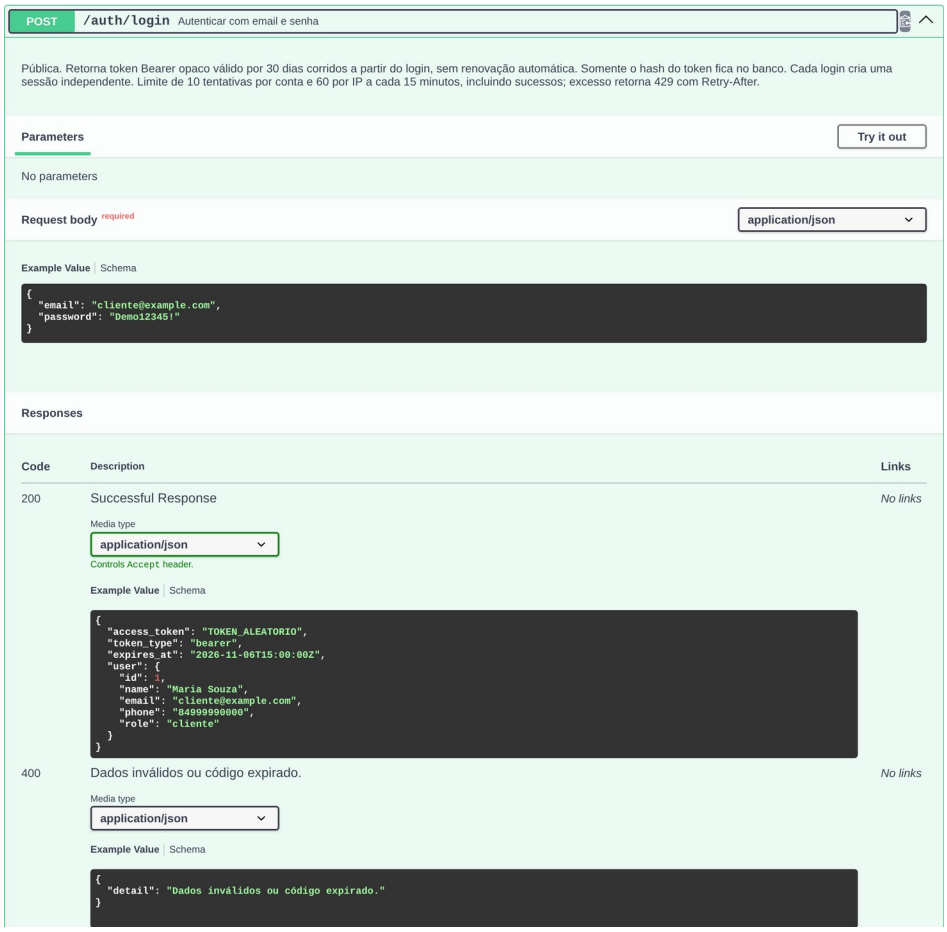


Figura 5 – Documentação da rota de login: descrição, corpo da requisição e exemplos de resposta de sucesso e de erro.

8 INTEGRAÇÃO DO APLICATIVO AO SERVIDOR

O aplicativo deixou de usar dados de exemplo e sessão em memória: todas as telas consultam e alteram os dados pelas rotas da API. A comunicação foi centralizada em um módulo reutilizável, `lib/core/api/api_client.dart`, usado por todas as telas:

- **Endereço da API centralizado**, definido na execução por `--dart-define=API_BASE_URL` (padrão `http://127.0.0.1:8000`). A versão de publicação só aceita um endereço HTTPS.
- **Serialização e desserialização JSON** em UTF-8 em um único lugar, com envio automático do cabeçalho `Authorization: Bearer` quando há sessão.
- **Tempo limite de 15 segundos** e conversão de cada erro em uma mensagem compreensível. Nos erros 422, a mensagem indica o campo (por exemplo, "Senha: Preencha este campo.").
- **Sessão expirada:** uma resposta 401 encerra a sessão local e leva o usuário de volta ao login.

Nas telas foram implementados os estados de **carregamento** (indicador de progresso enquanto a resposta não chega), **erro** (mensagem com opção de tentar novamente) e **confirmação**. Uma alteração só é exibida como concluída depois da resposta de sucesso do

servidor; enquanto isso, o botão fica desabilitado e o formulário é preservado. Em falhas de conexão, o aplicativo informa que o servidor está indisponível. Se uma alteração exceder o tempo limite, ele avisa que não conseguiu confirmar a ação e orienta o usuário a verificar antes de repetir, já que a operação pode ter sido gravada. No chat, a mensagem só é marcada como enviada quando o servidor confirma, pelo WebSocket, que ela foi gravada no banco.

Para o celular conectado por cabo USB, o comando `adb reverse tcp:8000 tcp:8000` faz o aparelho acessar a API do computador; no emulador Android, usa-se o endereço `http://10.0.2.2:8000`.

9 IDENTIFICAÇÃO E CONTROLE DE ACESSO

9.1 Cadastro e autenticação real

O login simulado da Avaliação 2 foi substituído por cadastro e autenticação reais no servidor. As senhas são armazenadas apenas como **hash bcrypt** (custo 12 e salt individual) e nunca ficam salvas no aplicativo. O login gera um token aleatório de 256 bits, válido por 30 dias; o banco guarda somente o hash SHA-256 desse token. No aparelho, o token fica no armazenamento seguro (`flutter_secure_storage`). O logout revoga a sessão no servidor. Há limite de tentativas de login e de cadastro, e o excesso retorna 429. Dados pessoais recuperáveis, como endereço e telefone, são cifrados no banco com **AES-GCM**.

9.2 Registros associados aos usuários

O dono de cada registro é obtido do token da sessão, e não do corpo da requisição. Por isso, um usuário não consegue criar dados em nome de outro. Ao criar uma solicitação, o servidor grava `client_id` com o usuário autenticado. Da mesma forma, mensagens recebem o remetente (`sender_id`), endereços o proprietário (`user_id`), e horários e perfil profissional ficam ligados ao profissional logado.

9.3 Permissões validadas no servidor

Tabela 8 – Permissões por perfil.

Perfil	O que pode consultar e alterar
Visitante (sem login)	Cadastro, login, categorias, busca de profissionais, perfil público, horários livres e avaliações públicas.
Cliente	Os próprios dados e endereços; criar solicitações; cancelar as próprias solicitações pendentes ou aceitas; avaliar serviços próprios concluídos; conversar nas próprias solicitações.
Profissional	Os próprios dados; manter perfil profissional e agenda; aceitar, recusar, iniciar e concluir somente as solicitações atribuídas a ele; conversar nessas solicitações.
Qualquer usuário	Nunca acessa solicitações, mensagens ou endereços de terceiros, nem dados privados de outros usuários.

9.4 Tentativas sem autenticação ou sem permissão

Cada rota privada valida a sessão antes de executar. O comportamento foi verificado com a API em execução:

- requisição a `GET /users/me` sem token → **401** `{"detail": "Autenticação necessária ou sessão expirada."}`;

- cliente acessando `GET /professionals/me`, rota exclusiva de profissional → **403** `{"detail": "Permissão insuficiente."}`;
- cliente tentando aceitar a própria solicitação → **403** `{"detail": "Seu perfil não pode executar esta etapa."}`;
- profissional tentando concluir uma solicitação ainda pendente → **409** `{"detail": "Transição de estado não permitida."}`;
- login com senha incorreta → **401** `{"detail": "Email ou senha inválidos."}`.

Um usuário que tenta acessar uma solicitação da qual não participa recebe 403, e o aplicativo trata essas respostas exibindo a mensagem correspondente sem expor detalhes internos.

10 COMO EXECUTAR

O passo a passo completo, para Windows, Linux e macOS, está no `README.md` do repositório. Em resumo, na pasta `backend`, com Python 3.12 ou superior:

```
python -m venv .venv
.venv/bin/pip install -r requirements.txt      # Windows: .venv\Scripts\pip
cp .env.example .env
.venv/bin/python -m app.setup_secrets
.venv/bin/python -m app.seed
.venv/bin/python -m uvicorn app.main:app --port 8000
```

Depois, na raiz do projeto, `flutter pub get` e `flutter run --dart-define=API_BASE_URL=http://10.0.2.2:8000` (emulador). Chaves, banco e configurações privadas não são versionados; os passos acima recriam tudo o que é necessário. O arquivo de configuração do Firebase é opcional: sem ele, o aplicativo compila e funciona, apenas sem notificações push.

11 RESULTADOS DA ETAPA

Com a Avaliação 3, o Zé Resolve passou de um protótipo com dados locais para um sistema distribuído completo: um aplicativo Flutter que consome uma API própria, com regras de negócio, persistência relacional e controle de acesso centralizados no servidor. O contrato da API está documentado em OpenAPI, permitindo que outras interfaces a utilizem sem consultar o código-fonte. O sistema foi executado a partir de um clone limpo do repositório, seguindo o README, e respondeu como descrito neste documento.

12 LINKS PARA ENTREGA

Repositório no GitHub: https://github.com/ThalysRD/Ze-Resolve

Documentação OpenAPI:
https://github.com/ThalysRD/Ze-Resolve/blob/main/docs/openapi.json

Guia da API REST: https://github.com/ThalysRD/Ze-Resolve/blob/main/docs/api.md

Arquitetura, modelo de dados e justificativas:
https://github.com/ThalysRD/Ze-Resolve/blob/main/docs/arquitetura.md

Protótipo no Figma: https://www.figma.com/design/OmP9I8Z8iEBPy6duwa1mZT/App-projeto-integrador-2?node-id=0-1&t=oDYtz27xaVNoQlmy-1

APÊNDICE A – ROTAS DA API

Lista completa das operações documentadas no OpenAPI. "Bearer" indica que a rota exige o token de sessão; as permissões específicas de cada uma estão descritas no Swagger.

Tabela 9 – Todas as operações da API.

Método	Caminho	Acesso	Descrição
GET	`/health`	Pública	Verificar disponibilidade
POST	`/auth/password/reset-request`	Pública	Solicitar código de recuperação
POST	`/auth/password/reset`	Pública	Redefinir senha com código de uso único
POST	`/auth/email/verification-request`	Pública	Solicitar confirmação de email
POST	`/auth/email/verify`	Pública	Confirmar email com código de uso único
POST	`/auth/register`	Pública	Cadastrar cliente ou profissional
POST	`/auth/login`	Pública	Autenticar com email e senha
POST	`/auth/logout`	Bearer	Revogar a sessão atual
GET	`/users/me/sessions`	Bearer	Consultar minhas sessões sem expor tokens
POST	`/users/me/sessions/revoke-all`	Bearer	Encerrar todas as minhas sessões
POST	`/users/me/closure`	Bearer	Encerrar minha conta e remover dados pessoais dos campos próprios
GET	`/users/me`	Bearer	Consultar meus dados
PATCH	`/users/me`	Bearer	Atualizar nome e telefone
POST	`/users/me/password`	Bearer	Alterar minha senha
GET	`/users/me/addresses`	Bearer	Listar meus endereços
POST	`/users/me/addresses`	Bearer	Adicionar endereço
PUT	`/users/me/addresses/{address_id}`	Bearer	Editar endereço
POST	`/users/me/addresses/{address_id}/archive`	Bearer	Arquivar endereço
GET	`/legal/documents/{kind}`	Pública	Consultar documento e versão de aceite
GET	`/categories`	Pública	Listar categorias
GET	`/professionals/me`	Bearer	Consultar minha base de atendimento
PUT	`/professionals/me`	Bearer	Criar ou substituir meu perfil profissional
GET	`/professionals`	Pública	Buscar profissionais por categoria e proximidade
GET	`/professionals/{professional_id}`	Pública	Consultar perfil público
GET	`/professionals/me/availability`	Bearer	Consultar meus horários publicados
POST	`/professionals/me/availability`	Bearer	Publicar horário de atendimento

Método	Caminho	Acesso	Descrição
POST	`/professionals/me/availability/{slot_id}/archive`	Bearer	Retirar horário livre da agenda
GET	`/professionals/{professional_id}/availability`	Pública	Consultar horários livres de um prestador
POST	`/requests`	Bearer	Solicitar um serviço
GET	`/requests`	Bearer	Listar minhas solicitações
GET	`/requests/summary`	Bearer	Resumo dos meus atendimentos
GET	`/requests/{request_id}`	Bearer	Consultar detalhes de uma solicitação
PATCH	`/requests/{request_id}/status`	Bearer	Alterar etapa do serviço
POST	`/requests/{request_id}/review`	Bearer	Avaliar serviço concluído
GET	`/professionals/{professional_id}/reviews`	Pública	Listar avaliações públicas
GET	`/requests/{request_id}/messages`	Bearer	Consultar conversa do serviço
POST	`/requests/{request_id}/messages`	Bearer	Enviar mensagem
GET	`/request-events`	Bearer	Listar meus avisos de agendamento
POST	`/request-events/{event_id}/read`	Bearer	Marcar meu aviso de agendamento como lido
PUT	`/users/me/push-device`	Bearer	Cadastrar dispositivo para push
GET	`/users/me/notification-settings`	Bearer	Consultar disponibilidade de push
GET	`/notifications`	Bearer	Consultar mensagens recebidas
POST	`/notifications/{message_id}/read`	Bearer	Marcar mensagem recebida como lida